

COMPUTER VISION LAB 03: MASTER CHEAT-SHEET

Complete Beginner-to-Advanced Exam Guide | Geometric Transformations | FAST-NUCES

1. Core Theoretical Foundations & Detailed Concepts

This section establishes the absolute mathematical and conceptual groundwork for Geometric Transformations. If you have zero background in Computer Vision, read every definition carefully – these core concepts form the basis for all exam MCQs and coding logic.

1.1 Geometric vs. Photometric Transformations

In digital image processing, operations are strictly divided into two distinct categories based on what part of the image function is being altered:

1. Geometric Transformation: A mathematical mapping operation that alters the **spatial position, orientation, scale, or shape** of an object in a digital image without modifying its underlying pixel intensity or color values.

- *Mathematical Notation:* $(x, y) \rightarrow (x', y')$
- *Core Mechanics:* It changes **where** the pixel is located on the 2D spatial grid.
- *Real-World Analogy:* Cutting out a photo printed on a flexible rubber sheet and stretching, spinning, sliding, or slanting the sheet. The color painted on the sheet doesn't change, but its location on the wall moves.
- *Key Examples:* Translation (sliding), Rotation (spinning), Scaling (zooming), Shearing (slanting), Rigid Transformation, Affine Warping, Projective Transformation (Homography).

2. Photometric Transformation: An operation that modifies pixel color, brightness, contrast, or intensity values without altering spatial coordinates or the structure of the pixel grid.

- *Mathematical Notation:* $I(x, y) \rightarrow I'(x', y')$
- *Core Mechanics:* It changes **what** the pixel looks like.
- *Real-World Analogy:* Adjusting the brightness knob on a television screen. The actors stay in the exact same spot on the screen, but their clothes appear brighter or darker.
- *Key Examples:* Brightness adjustment, contrast stretching, thresholding, histogram equalization, gamma correction, color space conversion (BGR to Grayscale).

GOLDEN AXIOM FOR MCQS

EXAM GOLDEN RULE (Axiom):

Geometry changes WHERE the pixel is located. Photometry changes WHAT the pixel looks like.

If an exam question asks: 'Which transformation modifies pixel coordinates while keeping intensity values invariant?', the answer is strictly **Geometric Transformation**.

1.2 Exhaustive Terminology Dictionary (24 Key Terms)

Term	Complete Beginner Explanation
Geometric Transformation	Spatial coordinate mapping $(x, y) \rightarrow (x', y')$ that changes pixel grid positions while preserving pixel intensity/color data.
Photometric Transformation	Pixel intensity mapping $I(x, y) \rightarrow I'(x', y')$ that alters color/brightness values while preserving spatial locations.
Image Coordinate System	A 2D spatial grid convention where the origin (0,0) sits at the top-left corner , with x running horizontally rightwards (width) and y running vertically downwards (height).
Linear Transformation	A subset of geometric mapping relying strictly on 2×2 matrix multiplication ($p' = A \cdot p$). It can scale, rotate, reflect, and shear, but is permanently anchored at the origin (0,0) .
Anchored Origin Limitation	The mathematical rule that multiplying any 2×2 linear matrix by the zero origin vector yields zero ($A \cdot [0,0]^T = [0,0]^T$), making translation (sliding) mathematically impossible with 2×2 matrices.
Translation	Sliding an image along x and/or y axes by fixed pixel displacement offsets (t_x, t_y) .
Affine Transformation	A geometric mapping combining a linear transformation and a translation ($p' = A \cdot p + b$). It strictly preserves collinearity and parallelism .
Collinearity	The geometric property where points lying on a single straight line in original space remain on a straight line after transformation, preserving relative distance ratios.
Parallelism	The geometric property where lines that are parallel in original space remain strictly parallel after transformation and never intersect.
Homogeneous Coordinates	A coordinate system that appends an extra dummy coordinate $w = 1$ to 2D points $([x, y, 1]^T)$, allowing translation to be represented via 3×3 matrix multiplication.
Matrix Composition	Multiplying multiple transformation matrices together into a single master matrix ($M_{\text{master}} = M_3 \cdot M_2 \cdot M_1$) so graphics hardware can execute complex multi-step transformations in a single pass per pixel.
Basis Vectors	Unit vectors $\hat{i} = [1, 0]^T$ (horizontal X-axis) and $\hat{j} = [0, 1]^T$ (vertical Y-axis) that define a coordinate grid. Matrix columns represent the new landing coordinates of these basis vectors.
Shearing	A transformation that slants/skews the image axes by shifting coordinates proportionally to another axis (e.g., $x' = x + k \cdot y$).
Rigid Transformation (Euclidean)	An affine mapping consisting only of rotation and translation (scale = 1.0), preserving distances, angles, and shapes.
Similarity Transformation	An affine mapping combining rotation, translation, and uniform isotropic scaling (preserves shape, changes size).
Projective Transformation (Homography)	A 3D-to-2D perspective mapping represented by a 3×3 matrix H that allows parallel lines to converge, simulating 3D perspective depth.
Denominator Effect (Perspective Division)	Normalization in projective geometry where x' and y' are divided by $w' = h_{31} \cdot x + h_{32} \cdot y + h_{33}$, causing distant points to shrink and create visual foreshortening.
Vanishing Point	The point on an image plane where parallel lines appear to converge under perspective projection.
Planar Alignment	Using homography to warp a trapezoidal perspective view of a flat surface (e.g., a document photographed at an angle) back into a flat, rectilinear rectangle.
cv2.warpAffine	OpenCV function that applies a 2×3 transformation matrix to warp an image.
cv2.warpPerspective	OpenCV function that applies a 3×3 homography matrix to warp an image.
cv2.getRotationMatrix2D	OpenCV API that calculates a 2×3 affine rotation matrix given a rotation center, angle, and scale factor.
cv2.getPerspectiveTransform	OpenCV API computing a 3×3 homography matrix from 4 corresponding source-destination point pairs.

<code>cv2.getAffineTransform</code>	OpenCV API computing a 2×3 affine matrix from 3 non-collinear corresponding source-destination point pairs.
-------------------------------------	---

1.3 Taxonomy & Hierarchy of Geometric Transformations

Geometric transformations are organized hierarchically based on how many geometric properties (invariants) they preserve. As you move down the hierarchy, transformations become more restrictive (preserving more properties); as you move up, they become freer (allowing more distortion).

Level	Matrix Representation	Preserved Invariants	Key Visual Characteristic
1. Projective (Homography)	3×3 Matrix (8 DOF)	Collinearity ONLY	Parallel lines CONVERGE at vanishing points. Models 3D perspective foreshortening.
2. Affine	2×3 Matrix (6 DOF)	Collinearity + Parallelism	Parallel lines REMAIN PARALLEL. Preserves ratios of distances along straight lines.
3. Similarity	2×3 Matrix (4 DOF)	Angles + Shape Ratios	Rotation + Translation + Uniform Scaling ($scale_x = scale_y$). Object shape is preserved, size changes.
4. Rigid (Euclidean)	2×3 Matrix (3 DOF)	Distances + Angles + Shapes	Rotation + Translation ONLY ($scale = 1.0$). Object is treated as a rigid solid body.

2. Coordinate Systems & Matrix Mechanics

2.1 Image Coordinate System vs. Cartesian Coordinates

A major trap for beginners is confusing standard Cartesian mathematics with the computer vision image coordinate system:

- **Standard Cartesian Math:** The origin (0,0) is at the **bottom-left corner**. Positive x points right; positive y points **UPWARDS**.
- **OpenCV Image Coordinates:** The origin (0,0) is at the **TOP-LEFT CORNER**. Positive x points **RIGHT** (columns, 0 to width-1); positive y points **DOWNWARDS** (rows, 0 to height-1).
- **Array Indexing Trap:** Spatial coordinates are written as $(x, y) = (\text{column}, \text{row})$, but NumPy matrix elements are accessed as `image[y, x] = image[row, column]`! Dimensions are obtained via `height, width = image.shape[:2]`.

2.2 Anatomy of a 2×2 Transformation Matrix & Basis Vectors

A 2×2 matrix represents a linear transformation: $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$.

To understand what this matrix does without getting lost in algebra, read it **VERTICALLY AS COLUMN VECTORS**:

- **Basis Vectors:** Standard 2D space is built from two fundamental unit vectors: $\hat{i} = [1, 0]^T$ (unit vector on X-axis) and $\hat{j} = [0, 1]^T$ (unit vector on Y-axis).
- **First Column $[a, c]^T$:** Tells you the exact new landing spot of the X-axis basis vector $\hat{i}$.
- **Second Column $[b, d]^T$:** Tells you the exact new landing spot of the Y-axis basis vector $\hat{j}$.

Variable	Structural Role in Basis Vectors	Visual & Geometric Effect
a	X-coordinate of new X-axis ($\hat{i}$)	Horizontal Scaling: Values > 1 stretch grid horizontally; negative values reflect across Y-axis.
c	Y-coordinate of new X-axis ($\hat{i}$)	Vertical Shearing: Tilts vertical grid lines up or down, breaking perpendicularity.
b	X-coordinate of new Y-axis ($\hat{j}$)	Horizontal Shearing: Tilts horizontal grid lines left or right ($x' = x + k \cdot y$).

d	Y-coordinate of new Y-axis ($\hat{j}$)	Vertical Scaling: Values > 1 stretch grid vertically; negative values reflect across X-axis.
---	--	--

3. Mathematical Equations & Formal Proofs

3.1 The Anchored Origin Limitation (Mathematical Proof)

Analytical Question: Can a standard 2×2 matrix translate (slide) a point, such as moving (10, 20) to (110, 70)?

Mathematical Proof: Multiply any arbitrary 2×2 linear matrix A by the origin zero vector $[0, 0]^T$:

$$[a, b; c, d] \cdot [0, 0]^T = [a(0) + b(0); c(0) + d(0)] = [0, 0]^T$$

Conclusion: $A \cdot [0, 0]^T = [0, 0]^T$ always. The origin is **permanently anchored at (0,0)**. A 2×2 matrix can spin, stretch, and shear space around (0,0), but it **CANNOT slide the origin across space**. Translation is mathematically impossible using 2×2 matrix multiplication alone.

3.2 Homogeneous Coordinates & Matrix Composition

The Hardware Bottleneck: Standard affine transformations require vector addition: $p' = A \cdot p + b$. This forces GPUs to perform matrix multiplication ($A \cdot p$), stop the hardware pipeline, and perform separate vector addition ($+ b$), creating a massive bottleneck during millions of parallel coordinate calculations per second.

The Homogeneous Solution: By adding a dummy coordinate $w = 1$ to 2D points $[x, y, 1]^T$, we upgrade to a 3×3 matrix that folds translation (tx, ty) directly into the rightmost column:

$$[x'; y'; 1] = [1, 0, tx; 0, 1, ty; 0, 0, 1] \cdot [x; y; 1] = [x + tx; y + ty; 1]$$

Power of Matrix Composition: Because scaling, rotation, and translation are all formatted as 3×3 matrix multiplications, a computer can multiply them into a single **master matrix** ($M_{\text{master}} = M_{\text{trans}} \cdot M_{\text{rot}} \cdot M_{\text{scale}}$) before touching geometry, executing complex multi-step transformations in a **single pass per pixel!**

3.3 Projective Geometry & The Denominator Effect

Projective transformations (Homographies) use a full 3×3 matrix H acting on homogeneous coordinates:

$$[x''; y''; w'] = [h_{11}, h_{12}, h_{13}; h_{21}, h_{22}, h_{23}; h_{31}, h_{32}, h_{33}] \cdot [x; y; 1]$$

To map back to 2D screen coordinates, divide by the third coordinate w' :

$$x' = (h_{11} \cdot x + h_{12} \cdot y + h_{13}) / (h_{31} \cdot x + h_{32} \cdot y + h_{33})$$

$$y' = (h_{21} \cdot x + h_{22} \cdot y + h_{23}) / (h_{31} \cdot x + h_{32} \cdot y + h_{33})$$

The Denominator Effect: In Affine matrices, the bottom row is strictly $[0, 0, 1]$, so $w' = 1$ everywhere (uniform scaling). In Projective matrices, the denominator $w' = h_{31} \cdot x + h_{32} \cdot y + h_{33}$ fluctuates dynamically based on spatial location (x, y) . As points move further away in virtual space, w' grows, causing x' and y' to shrink closer together — generating **3D depth, foreshortening, and vanishing points!**

3.4 Master Mathematical Formula Sheet

Name	Matrix Form	Algebraic Equations	Meaning & Usage
Linear Transformation	$x' = a \cdot x + b \cdot y, y' = c \cdot x + d \cdot y$	$p' = A \cdot p$	Standard 2x2 matrix multiplication anchored at origin.
2D Scaling Matrix	$[sx, 0; 0, sy]$	$x' = sx \cdot x, y' = sy \cdot y$	Diagonal matrix scaling axes independently.
2D Rotation Matrix	$[\cos(\theta), -\sin(\theta); \sin(\theta), \cos(\theta)]$	$x' = x \cos\theta - y \sin\theta, y' = x \sin\theta + y \cos\theta$	Rotates space counter-clockwise by angle θ around origin.
2D Reflection (X-axis)	$[1, 0; 0, -1]$	$x' = x, y' = -y$	Inverts Y-values, reflecting geometry across X-axis.
Horizontal Shear (X-shear)	$[1, kx; 0, 1]$	$x' = x + kx \cdot y, y' = y$	Slants grid horizontally proportionally to y.
Affine System	$p' = A \cdot p + b$	$[x'; y'] = [a, b; c, d][x; y] + [tx; ty]$	Linear transformation plus translation vector.
3x3 Homogeneous Affine	$[1, 0, tx; 0, 1, ty; 0, 0, 1]$	$[x'; y'; 1] = M \cdot [x; y; 1]$	Encodes translation as matrix multiplication.
Projective Homography	$x' = \text{Num_X} / \text{Denom}, y' = \text{Num_Y} / \text{Denom}$	$\text{Denom} = h31 \cdot x + h32 \cdot y + h33$	Perspective projection with coordinate-dependent division.

4. Practical Code Programs & Line-by-Line Breakdown

This section provides exam-ready Python code for every geometric transformation technique in Lab 03. Every code block is heavily commented and accompanied by a parameter breakdown table, input/processing/output flow, and modification guide.

4.1 Program 1: Image Rotation around Center

```
import cv2
import matplotlib.pyplot as plt
# 1. Read input image from disk using OpenCV
image = cv2.imread("image.jpg")
if image is None:
    print("Error: image.jpg not found")
else:
    # 2. Extract height and width (Note shape order: height, width)
    height, width = image.shape[:2]
    # 3. Compute rotation center in spatial coordinates (x, y) = (col, row)
    center = (width // 2, height // 2)
    # 4. Set rotation parameters
    angle = 45 # Angle in degrees (+ = counter-clockwise in OpenCV)
    scale = 1.0 # 1.0 scale keeps size invariant (Rigid transformation)
    # 5. Calculate 2x3 Affine rotation matrix
    M = cv2.getRotationMatrix2D(center, angle, scale)
    # 6. Apply affine transformation to warp the image
    rotated = cv2.warpAffine(image, M, (width, height))
    # 7. Convert BGR to RGB for correct plotting in Matplotlib
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    rotated_rgb = cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB)
    # 8. Render side-by-side comparison
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Before: Original Image")
    plt.axis("off")
    plt.subplot(1, 2, 2)
    plt.imshow(rotated_rgb)
    plt.title("After: Rotation (45 deg)")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```

Parameter	Type / Range	Exam Meaning & Effect
center	(x, y) tuple	Pivot point of rotation. (width//2, height//2) rotates around center.
angle	float (degrees)	Rotation angle. Positive = Counter-Clockwise; Negative = Clockwise.
scale	float (>0)	Isotropic scale factor. 1.0 = no zoom; 2.0 = double size; 0.5 = half size.

4.2 Program 2: Horizontal Shearing (X-Shear) with Canvas Expansion

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Read input image
image = cv2.imread("image.jpg")
if image is None:
    print("Error: image.jpg not found")
else:
    height, width = image.shape[:2]
    # 2. Define horizontal shearing factor kx
    kx = 0.5
    # 3. Create 2x3 X-Shear matrix (Data type MUST be float32)
    # Equations: x' = x + kx * y, y' = y
    M = np.float32([
        [1, kx, 0],
        [0, 1, 0]
    ])
    # 4. EXAM TRICK: Calculate expanded canvas width to prevent right border clipping!
    new_width = int(width + abs(kx) * height)
    # 5. Apply shear warping
    sheared = cv2.warpAffine(image, M, (new_width, height))
    # 6. Plotting
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    sheared_rgb = cv2.cvtColor(sheared, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Before: Original Image")
    plt.axis("off")
    plt.subplot(1, 2, 2)
    plt.imshow(sheared_rgb)
    plt.title("After: X-Axis Shearing")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```

Parameter	Type / Range	Exam Meaning & Effect
M	2x3 float32 array	Transformation matrix $\begin{bmatrix} 1 & kx & 0 \\ 0 & 1 & 0 \end{bmatrix}$. Must be float32, not int!
kx	float	Horizontal shear constant. $x' = x + kx * y$.
new_width	int	Expanded canvas width: $width + \text{int}(\text{abs}(kx) * \text{height})$. Prevents clipping!

4.3 Program 3: Perspective Transformation (Homography)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Read input image
image = cv2.imread("image.jpg")
if image is None:
    print("Error: image.jpg not found")
else:
    height, width = image.shape[:2]
    # 2. Define 4 source corner points (Top-Left, Top-Right, Bottom-Right, Bottom-Left)
    src_points = np.float32([
        [0, 0],
        [width - 1, 0],
        [width - 1, height - 1],
        [0, height - 1]
    ])
    # 3. Define 4 target perspective points (warping corners inward to simulate perspective)
    dst_points = np.float32([
        [100, 50],
        [width - 150, 0],
        [width - 50, height - 80],
        [50, height - 20]
    ])
    # 4. Compute 3x3 Homography Matrix from 4 point pairs
    M = cv2.getPerspectiveTransform(src_points, dst_points)
    # 5. Apply 3x3 perspective warp
    perspective = cv2.warpPerspective(image, M, (width, height))
    # 6. Plotting
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    perspective_rgb = cv2.cvtColor(perspective, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Before: Original Image")
    plt.axis("off")
    plt.subplot(1, 2, 2)
    plt.imshow(perspective_rgb)
    plt.title("After: Perspective Transformation")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```

Parameter / Function	Type / Requirement	Exam Meaning & Effect
src_points	4x2 float32 array	4 corner coordinates from original image plane.
dst_points	4x2 float32 array	4 corresponding corner coordinates on target perspective plane.
cv2.getPerspectiveTransform	API function	Computes 3x3 matrix H. Requires EXACTLY 4 point pairs!
cv2.warpPerspective	API function	Warpes image using 3x3 matrix H and dynamic perspective division.

5. Quick Reference & Exam Survival Kit

5.1 Master Comparison: Linear vs. Affine vs. Projective

Feature / Property	Linear	Affine	Projective (Homography)
Matrix Dimensions	2x2 Matrix	2x3 (OpenCV) / 3x3 (Math)	3x3 Homography Matrix
Translation Supported?	NO ($A \cdot [0,0]^T = [0,0]^T$)	YES (via $w = 1$)	YES (via $w = 1$)
Preserves Collinearity?	YES	YES	YES
Preserves Parallelism?	YES	YES (Parallel lines stay parallel)	NO (Parallel lines CONVERGE)
Point Pairs Required	N/A	3 non-collinear point pairs	4 point pairs
OpenCV Warping API	<code>cv2.warpAffine()</code>	<code>cv2.warpAffine()</code>	<code>cv2.warpPerspective()</code>
OpenCV Matrix Generator	N/A	<code>cv2.getAffineTransform()</code>	<code>cv2.getPerspectiveTransform()</code>

5.2 MCQ Traps & Exam Trick Alerts

- **Trap 1: Linear Matrix Origin Anchor:** Linear 2x2 matrices CANNOT translate geometry because $A \cdot [0,0]^T = [0,0]^T$ always. If an MCQ asks how to perform translation using matrix multiplication alone, the answer is Homogeneous Coordinates (3x3).
- **Trap 2: Why GPUs use Homogeneous Math:** Homogeneous coordinates turn vector addition ($A \cdot p + b$) into pure matrix multiplication. This allows Matrix Composition (multiplying matrices together beforehand), enabling GPUs to execute multi-step warps in a single pass per pixel.
- **Trap 3: Parallelism Preservation Rule:** Affine transformations ALWAYS preserve parallelism (parallel lines stay parallel). Projective transformations DO NOT preserve parallelism (parallel lines converge at vanishing points).
- **Trap 4: Point Pair Counting Rule:** `cv2.getAffineTransform` requires EXACTLY 3 point pairs. `cv2.getPerspectiveTransform` requires EXACTLY 4 point pairs.
- **Trap 5: Matrix Dimensions in OpenCV:** `cv2.warpAffine` accepts a 2x3 matrix. `cv2.warpPerspective` accepts a 3x3 matrix.
- **Trap 6: Bottom Row Distinction:** In Affine matrices, the bottom row is strictly $[0, 0, 1]$ (denominator = 1). In Projective matrices, the bottom row is $[h31, h32, h33]$ (denominator varies dynamically with x and y).
- **Trap 7: Rigid vs. Similarity:** Rigid requires scale = 1.0 (preserves size and shape). Similarity allows scale $\neq 1.0$ (preserves shape, changes size).

5.3 WHICH FUNCTION DO I USE? (Decision Table)

Exam Task Requirement	Matrix Generator / Setup	Warping Execution API
Rotate an image around a specific pivot point	<code>cv2.getRotationMatrix2D(center, angle, scale)</code>	<code>cv2.warpAffine(img, M, (w, h))</code>
Translate (slide) an image by offset (tx, ty)	<code>M = np.float32([1, 0, tx; 0, 1, ty])</code>	<code>cv2.warpAffine(img, M, (w, h))</code>
Shear an image horizontally by factor kx	<code>M = np.float32([1, kx, 0; 0, 1, 0])</code>	<code>cv2.warpAffine(img, M, (new_w, h))</code>
Compute Affine warp from 3 point pairs	<code>cv2.getAffineTransform(src_3pts, dst_3pts)</code>	<code>cv2.warpAffine(img, M, (w, h))</code>
Un-warp a document / perspective projection from 4 points	<code>cv2.getPerspectiveTransform(src_4pts, dst_4pts)</code>	<code>cv2.warpPerspective(img, M, (w, h))</code>
Resize image with independent scale factors	<code>cv2.resize(img, None, fx=sx, fy=sy)</code>	Direct output image

5.4 COMMON CODING ERRORS & FIXES

Problem / Error Message	Root Cause	Exam Fix / Correct Syntax
<code>cv2.error: (-215) src.type() == CV_32F in warping</code>	Passing python list or int64 array as matrix M.	Explicitly define M as float32: <code>M = np.float32([...])</code>
Image border clipped/cut off during shearing	Keeping original image width in <code>warpAffine</code> .	Calculate expanded canvas: <code>new_w = int(w + abs(kx)*h)</code>
Image translates in opposite vertical direction	Assuming positive y goes UP like Cartesian math.	In OpenCV top-left origin, positive ty moves DOWNWARDS.
Incorrect point count in <code>getPerspectiveTransform</code>	Passing 3 points instead of 4.	Perspective requires EXACTLY 4 point pairs!

5.5 Last-Minute Revision Highlights Checklist

Origin: Top-left corner (0,0). x = right (cols), y = down (rows).

Linear 2x2 Matrix: Columns represent landing coordinates of basis vectors $\hat{i}$ and $\hat{j}$. Cannot translate!

Affine Matrix: 2x3 in OpenCV. Preserves collinearity and parallelism. Uses 3 point pairs.

Perspective Matrix (Homography): 3x3 matrix H. Preserves collinearity ONLY. Parallel lines converge. Uses 4 point pairs and perspective division.

Data Type Rule: All transformation matrices in OpenCV MUST be `np.float32`.

Matplotlib Color Rule: Convert BGR to RGB via `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before `plt.imshow()`.